

Mutual Exclusion with Busy Waiting

Part 1: interrupts, lock variables, and strict alternation

Definition

Mutual exclusion means making sure that while one process is inside a critical region using shared data, no other process can enter a conflicting critical region.

1. Purpose of Busy-Waiting Solutions

Main Idea

The goal is to prevent race conditions while processes update shared memory, shared files, or shared data structures.

Key Requirement

Only one process should be allowed inside the protected critical region at a time. Every proposal in this section is an attempt to enforce that rule.

Busy Waiting

Busy waiting means repeatedly checking a condition in a loop until it becomes true. A lock that uses busy waiting is called a **spin lock**.

Cost

Busy waiting wastes CPU time because the waiting process keeps running but does no useful work. It is only reasonable when the expected wait is very short.

2. Approach 1: Disabling Interrupts

How It Works

On a single-processor system, a process can disable interrupts just after entering its critical region and re-enable them just before leaving.

With interrupts disabled, clock interrupts cannot occur. Since a process switch normally happens because of clock or other interrupts, the current process keeps the CPU and can update shared memory without being interrupted.

Case	Effect
Single CPU	Disabling interrupts can prevent another process from running on that CPU.
User process	Dangerous, because a user program might disable interrupts and never enable them again.
Kernel code	Sometimes useful for a few instructions while updating kernel lists or variables.
Multicore system	Not enough, because other CPUs continue running and can still access shared memory.

Why It Is Not a General Solution

Giving user processes the power to turn off interrupts is unsafe. A broken or malicious program could stop the system from responding.

Kernel Use

Disabling interrupts can still be useful inside the operating system kernel for very short critical updates, such as modifying the ready-process list.

Multicore Limitation

On a multiprocessor or multicore machine, disabling interrupts on one CPU does not stop another CPU from accessing the same shared memory. More sophisticated synchronization is needed.

3. Approach 2: Lock Variables

Lock Variable

A **lock variable** is a shared variable used to indicate whether some process is already inside a critical region.

The simple idea is:

- `lock = 0`: no process is inside the critical region.
- `lock = 1`: some process is inside the critical region.

```
while (lock == 1){  
    // busy wait  
    /*loop tasks*/  
}  
lock = 1;  
  
critical_region();  
  
lock = 0;  
noncritical_region();
```

Why This Looks Reasonable

A process first waits until `lock` is 0. Then it sets `lock` to 1 and enters the critical region. When it leaves, it sets `lock` back to 0.

3.1 The Fatal Race

Step	Process A	Process B
1	Reads <code>lock = 0</code> .	–
2	Gets interrupted before setting <code>lock = 1</code> .	–
3	–	Reads <code>lock = 0</code> .
4	–	Sets <code>lock = 1</code> and enters the critical region.
5	Resumes and sets <code>lock = 1</code> .	Already inside the critical region.
6	Enters the critical region too.	Race condition occurs.

Fatal Flaw

Testing the lock and setting the lock are two separate operations. A process switch can happen between them, allowing two processes to enter the critical region.

Why Checking Twice Does Not Fix It

If the process checks the lock again just before storing into it, the race can still happen immediately after the second check. The operation must be atomic, not merely checked more often.

4. Approach 3: Strict Alternation

Strict Alternation

Strict alternation uses a shared variable `turn` to force two processes to take turns entering their critical regions.

Initially, `turn = 0`, so process 0 is allowed to enter first.

Process 0

```
while (TRUE) {  
    while (turn != 0){  
        /*loop tasks*/  
    }  
    critical_region();  
    turn = 1;  
    noncritical_region();  
}
```

Process 1

```
while (TRUE) {  
    while (turn != 1){  
        /*loop tasks*/  
    }  
    critical_region();  
    turn = 0;  
    noncritical_region();  
}
```

Notice the Semicolon

The empty statement after each inner `while` loop is intentional. The process repeatedly tests `turn` until it is allowed to continue.

5. Why Strict Alternation Fails

What It Gets Right

Strict alternation prevents two processes from being inside their critical regions at the same time, so it avoids the race condition.

What It Gets Wrong

It forces the processes to alternate even when one process is not interested in entering its critical region.

Suppose process 0 finishes its critical region and sets `turn = 1`. If process 1 is still doing noncritical work, process 0 cannot enter again. It must wait for process 1, even though process 1 is not using the shared data.

Problem	Explanation
Unnecessary waiting	A fast process may be blocked by a slow process doing unrelated noncritical work.
Wasted CPU time	The blocked process spins in a loop checking <code>turn</code> .
Violates condition 3	A process outside its critical region can block another process from entering.
Poor practical behavior	Neither process can enter twice in a row, even if the other process does not need the critical region.

Print Spooler Meaning

If critical regions are associated with writing the spooler directory, strict alternation would mean process 0 could not queue two print jobs in a row unless process 1 also took a turn.

6. Comparison of the First Three Attempts

Method	Main Benefit	Main Failure
Disabling interrupts	Simple on one CPU.	Unsafe for user processes and ineffective across multiple CPUs.
Lock variable	Simple software idea.	Testing and setting the lock are not atomic.
Strict alternation	Prevents simultaneous critical-region entry.	Blocks a process even when the other process is outside its critical region.

7. Quick Review

Key Points

- Busy waiting repeatedly tests a condition while consuming CPU time.
- A spin lock is a lock that uses busy waiting.
- Disabling interrupts can work briefly inside the kernel on a single CPU, but is not a general user-level solution.
- A simple lock variable fails because checking and setting the lock are separate operations.
- Strict alternation prevents races but forces processes to take turns unnecessarily.
- Strict alternation violates the requirement that a process outside its critical region must not block another process.